PROFESSORE DIGITALE · PER CHI SVILUPPA E DISTRIBUISCE

Guida agli aggiornamenti

Come preparare, numerare e pubblicare
nuove versioni e nuovi contenuti



Versione 2.1.1 · Pacchetti di contenuti · Repository APT firmato · GitHub Pages

Danilo D'Auria
Ideazione e sviluppo · Tecnico IT & Divulgatore Digitale per il Sociale

2.1
Settembre 2026

Indice

1 **Il quadro generale** 3
Tre tipi di aggiornamento e il percorso dal computer agli utenti

2 **Come si numerano le versioni** 4
Quando si passa da 2.1 a 2.2, e quando alla 3.0

3 **Preparare il laboratorio (una volta sola)** 6
Strumenti, sorgenti, GitHub, chiave di firma

4 **Ampliare domande e lezioni** 8
I pacchetti di contenuti, passo per passo

5 **Modificare il programma** 11
Dove si trova cosa, provare dai sorgenti, i test

6 **Pubblicare una nuova versione** 13
rilascio.sh, pubblica-sito.sh, pubblica-contenuti.sh

7 **Come gli utenti ricevono gli aggiornamenti** 15
Dall'app, dal Gestore aggiornamenti, dalla chiavetta

8 **Compatibilità e dati degli allievi** 17
Non perdere mai i progressi tra una versione e l'altra

9 **Migliorie consigliate fino alla 3.0** 18
Programma, contenuti, distribuzione: priorità e impegno

10 **Lista di controllo e problemi frequenti** 21
Da stampare e tenere accanto al computer

COSA È CAMBIATO NELLA 2.1

Questa guida accompagna la versione **2.1.1**, che introduce tutto ciò che serve per aggiornare il Professore Digitale in modo ordinato: i **pacchetti di contenuti**, il **controllo aggiornamenti con note e verifica SHA-256**, lo script di **rilascio automatico**, il **repository APT firmato** e la pubblicazione su **GitHub Pages**. Le versioni 2.1.0 e 2.1.1 sono state prodotte proprio con questi strumenti, come prova reale.

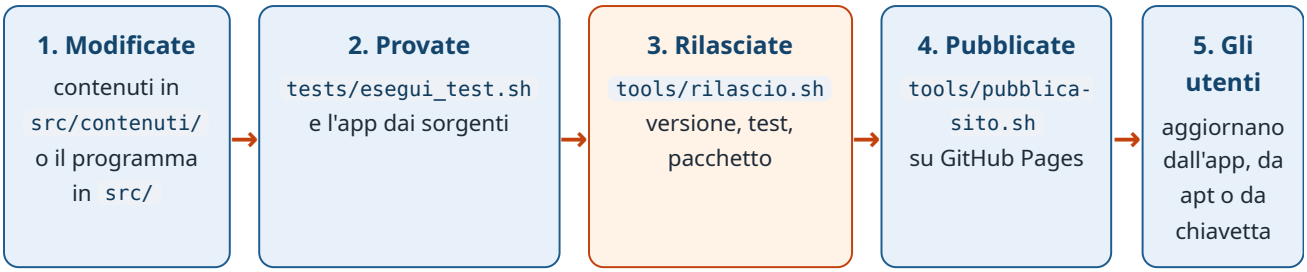
CAPITOLO 1

Il quadro generale

Un aggiornamento del Professore Digitale può riguardare tre cose diverse. Distinguerle bene fa risparmiare lavoro a voi e fatica agli utenti.

Che cosa cambia	Esempi	Cosa serve	Cosa fa l'utente
Contenuti domande e lezioni	30 nuove risposte, una lezione sulla PEC, una correzione in un quiz	Un **pacchetto di contenuti** (file JSON). Nessuna nuova versione del programma.	Un clic su **Aggiorna** in Impostazioni, oppure importa il file dalla chiavetta.
Programma	Una nuova funzione, un errore corretto, una grafica migliore	Una **nuova versione** del pacchetto `.deb`, preparata con `tools/rilascio.sh`.	Scarica e installa il nuovo pacchetto, oppure lo riceve dal Gestore aggiornamenti.
Documentazione	Vademecum, questa guida	Il PDF aggiornato in `docs/`, incluso alla versione successiva.	Lo apre dall'app (Impostazioni → Informazioni).

Il percorso di un aggiornamento



Il principio che non cambia

Il Professore Digitale **non si collega mai a Internet da solo** e **non installa niente da solo**. Tutti gli strumenti descritti qui rispettano questa regola: l'utente vede cosa è cambiato, decide lui e i file vengono verificati con un'impronta digitale (SHA-256) prima di essere usati. È il "protocollo etico di aggiornamento" dei documenti originali, ora realizzato in concreto.

> **IN PRATICA, NELLA MAGGIOR PARTE DEI CASI**
>
> Le novità più frequenti saranno **nuove domande**. Per quelle basta modificare un file di testo e lanciare `tools/pubblica-contenuti.sh`: niente nuova versione, niente reinstallazione.

CAPITOLO

2 Come si numerano le versioni

Il numero di versione è una promessa verso chi usa il programma. Il Professore Digitale segue la convenzione più diffusa nel software libero, il **versionamento semantico**: tre numeri separati da punti.

3.0.0

MAGGIORE

Cambiamenti profondi: nuova interfaccia, nuove modalità di lavoro, requisiti diversi. Si annuncia e si accompagna con istruzioni.

2.2.0

MINORE

Novità compatibili: nuove funzioni, nuove lezioni incluse, migliorie. Tutto ciò che c'era continua a funzionare.

2.1.2

CORREZIONE

Solo errori corretti e piccoli ritocchi. Nessuna novità da imparare.

Situazione	Comando	Esempio
Un quiz aveva la risposta sbagliata; un testo tagliato sul telefono	`tools/rilascio.sh correzione "…"`	2.1.1 → 2.1.2
Nuova sezione "Dispense stampabili"; nuove lezioni incluse nel pacchetto	`tools/rilascio.sh minore "…"`	2.1.2 → 2.2.0
Modalità aula in rete, profili allievi, nuova grafica	`tools/rilascio.sh maggiore "…"`	2.4.0 → 3.0.0
Solo nuove domande o lezioni in un pacchetto	Nessuna nuova versione del programma: si aumenta la **versione del pacchetto** e si usa `tools/pubblica-contenuti.sh`	pacchetto 1.0.0 → 1.1.0

Dove si trova il numero

Il numero di versione del programma sta in **un solo posto**: il file `VERSION` nella cartella del progetto. Programma, finestra "Informazioni", pacchetto `.deb`, manuale `man` e scheda del Software Center lo leggono da lì. Non va mai modificato a mano: lo aggiorna `tools/rilascio.sh`, insieme al registro delle modifiche `packaging/changelog`.

Le versioni dei pacchetti di contenuti

Ogni pacchetto di contenuti ha una sua versione, indipendente da quella del programma, con la stessa logica: **1.0.1** per correggere un refuso, **1.1.0** per aggiungere domande, **2.0.0** se lo riorganizzate da capo. L'app confronta la versione installata con quella pubblicata e propone **Aggiorna** solo quando serve.

Il campo facoltativo `richiede_app` indica la versione minima del programma necessaria: un pacchetto con `"richiede_app": "2.1.0"` viene rifiutato con una spiegazione chiara da un Professore più vecchio, invece di causare errori.

> **REGOLA D'ORO**
> Un numero pubblicato non si riusa mai. Se la 2.2.0 aveva un errore, si pubblica la 2.2.1: mai una "2.2.0 corretta", perché i computer che hanno già la 2.2.0 non la riceverebbero.

CAPITOLO

3 Preparare il laboratorio

Queste operazioni si fanno una volta sola, sul computer da cui preparerete gli aggiornamenti. Va bene qualsiasi PC con Debian, Ubuntu, Mint o FeniceOS e una connessione a Internet.

1. Gli strumenti

```
sudo apt install git python3 python3-venv nodejs dpkg-dev apt-utils gnupg lintian curl
python3 -m venv ~/pd-strumenti && ~/pd-strumenti/bin/pip install playwright
~/pd-strumenti/bin/playwright install --with-deps chromium     # browser per i test
automatici
```

Per usare i test, attivate l'ambiente prima di lavorare: `source ~/pd-strumenti/bin/activate`.

2. I sorgenti

Estraete l'archivio dei sorgenti ricevuto e aprite un terminale nella cartella. È già un **archivio git** con la storia delle versioni 2.0.0, 2.1.0 e 2.1.1 (i "tag" `v2.0.0`, `v2.1.0`, `v2.1.1`).

```
tar xzf professore-digitale-2.1.1-sorgenti.tar.gz && cd professore-digitale
git log --oneline --decorate        # la storia del progetto
```

3. GitHub: casa dei sorgenti e sito degli aggiornamenti

1. Create un account gratuito su **github.com** (ad esempio `danilodauria`) e un repository pubblico chiamato `professore-digitale`.
2. Inviate i sorgenti: `git remote add origin https://github.com/NOMEUTENTE/professore-digitale.git` e poi `git push -u origin main --tags`.
3. Dopo la prima pubblicazione (capitolo 6), su GitHub aprite **Settings → Pages**, scegliete **Deploy from a branch**, ramo **gh-pages**, cartella **/ (root)**. Il sito sarà `https://NOMEUTENTE.github.io/professore-digitale`.

4. Il file rilascio.conf

È l'unico file da adattare. Indica dove verranno pubblicati gli aggiornamenti; il suo contenuto viene scritto dentro il pacchetto, così ogni Professore installato sa dove guardare quando l'utente preme "Controlla aggiornamenti".

```
BASE_URL="https://danilodauria.github.io/professore-digitale"
AGGIORNAMENTI_URL="$BASE_URL/aggiornamenti.json"
PAGINA_PROGETTO="https://github.com/danilodauria/professore-digitale"
MANUTENTORE="Danilo D'Auria <danilodauria.site@gmail.com>"
REPO_GIT="https://github.com/danilodauria/professore-digitale.git"
```

SE IL VOSTRO NOME UTENTE GITHUB È DIVERSO

Correggete gli indirizzi **prima** di distribuire la prossima versione. Le installazioni 2.0.0 cercano il file `version.txt` in `https://raw.githubusercontent.com/danilodauria/professore-digitale/main/` : se il repository ha un altro indirizzo, quegli utenti dovranno installare a mano la 2.1 una volta sola. Dalla 2.1 in poi l'indirizzo viene da `rilascio.conf` .

5. La chiave di firma (solo per il repository APT)

```
tools/repo-apt.sh --crea-chiave
gpg --export-secret-keys --armor "danilodauria.site@gmail.com" > chiave-privata.asc
```

La chiave privata dimostra che gli aggiornamenti vengono davvero da voi. Conservate `chiave-privata.asc` su una chiavetta tenuta al sicuro, **mai** su GitHub. Se perdete la chiave dovrete crearne una nuova e far reinstallare il repository a tutti.

CAPITOLO

4 Ampliare domande e lezioni

Le domande della guida interna e le lezioni si ampliano con i **pacchetti di contenuti**: semplici file di testo in formato JSON, con i nomi dei campi in italiano. Non serve saper programmare, basta attenzione a virgolette e virgole.

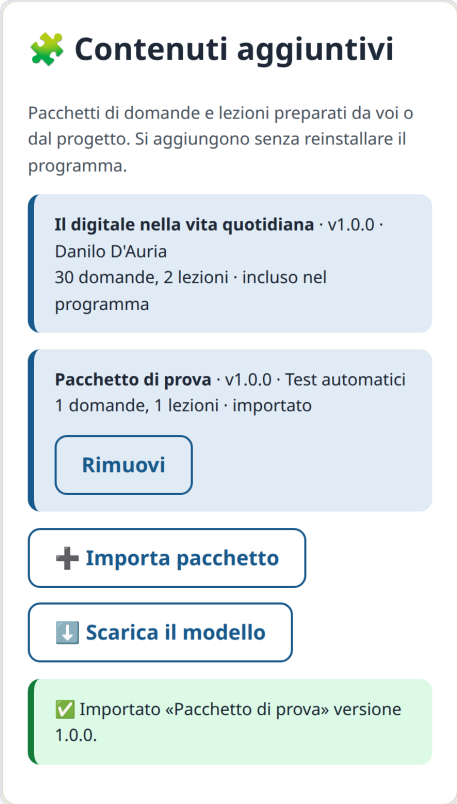


Figura 1. Impostazioni → **Contenuti aggiuntivi**: i pacchetti inclusi nel programma, quelli importati (rimovibili), i pulsanti per importare un file e per scaricare il modello da compilare.

Gli stessi pacchetti si gestiscono dal terminale:

```
professore-digitale-contenuti modello > mie.json
professore-digitale-contenuti valida mie.json
professore-digitale-contenuti installa mie.json
sudo professore-digitale-contenuti installa mie.json --sistema
professore-digitale-contenuti elenco
professore-digitale-contenuti rimuovi mie-domande
```

Com'è fatto un pacchetto

```
{
 "formato": "professore-digitale-contenuti",  ← sempre uguale
 "formato_versione": 1,                       ← sempre 1
 "id": "posta-certificata",                   ← minuscole, cifre, trattini
 "titolo": "La PEC spiegata semplice",
 "versione": "1.0.0",
 "autore": "Danilo D'Auria",
 "descrizione": "8 domande e una lezione sulla posta certificata.",
 "richiede_app": "2.1.0",
 "suggerimenti": ["Cos'è la PEC?"],            ← pulsanti rapidi in «Chiedi»
 "domande": [ … ],
 "lezioni": [ … ]
}
```

Una domanda

Campo	Cosa scrivere
`domanda` *	La domanda come la farebbe un allievo: «Cos'è la PEC?»
`parole_chiave` *	Da 3 a 8 parole o brevi frasi con cui la gente la chiede davvero: `["pec", "posta certificata", "raccomandata elettronica"]`. Sono la parte più importante per farla trovare.
`risposta` *	La spiegazione. Si possono usare `**grassetto**`, `*corsivo*`, elenchi con «- » o «1. » e righe vuote (`\n\n`) per i paragrafi.
`schema`	Uno schema Mermaid facoltativo, disegnato sulla lavagna (vedi sotto).
`lezione`	L'`id` di una lezione collegata: sotto la risposta comparirà «Apri la lezione».

Una lezione

`id`, `titolo` *	Se l'`id` coincide con quello di una lezione esistente (es. `truffe`), la **sostituisce**: utile per correggerla o ampliarla senza nuova versione.
`icona`, `livello`, `minuti`, `obiettivo`	Emoji, gruppo nell'elenco («Primi passi», «Sicurezza», «Extra»...), durata, scopo in una frase.
`passi` *	Da 1 a 30 passi, ognuno con `titolo`, `testo`, e facoltativi `schema` e `consiglio`.
`quiz`	Domande con `opzioni` (2–6), `corretta` (**contando da 0**: la prima opzione è 0) e `spiegazione`.
`palestra`	Esercizio collegato: `click`, `dblclick`, `drag` o `typing`.

* campi obbligatori

Gli schemi alla lavagna

Gli schemi usano il linguaggio **Mermaid**. Per restare leggibili su tutti i computer seguite tre regole: **massimo 5-6 riquadri**, testi **sempre tra virgolette**, direzione `LR` (orizzontale) per le sequenze brevi e `TD` (verticale) per le altre. Nel file JSON gli a capo si scrivono `\n` e le virgolette interne `\"`:

```
"schema": "flowchart LR\n  A[\"Apro la PEC\"] --> B[\"Scrivo\"] --> C[\"Invio\"] --> D[\"Ricevuta\"]"
```

Scrivere buone risposte

- **Brevi**: 40–120 parole. Il Professore le legge ad alta voce.
- **Una metafora** della vita quotidiana per i concetti astratti (l'armadio, la strada, la chiave).
- **Passi numerati** per le procedure; un'azione per passo.
- **Prudenza** su soldi, salute, leggi e scadenze: rimandate sempre ai canali ufficiali.

- **Niente dati che invecchiano** (prezzi, costi, nomi di offerte): la guida deve restare vera per anni.

Il ciclo di lavoro

1. Scaricate il modello (dall'app o con `professore-digitale-contenuti modello`) e compilatelo con un editor di testo (Mousepad, Kate, gedit, VS Code...).
2. Controllatelo: `professore-digitale-contenuti valida mie.json`. Gli errori indicano il punto esatto, ad esempio `domande[3].parole_chiave: elenco di 1-40 parole`.
3. Importatelo nell'app e fate le domande come le farebbero gli allievi. Se una risposta non viene trovata, aggiungete parole chiave.
4. Per **distribuirlo a tutti**: copiatelo in `src/contenuti/` del progetto e pubblicatelo (capitolo 6). Per **un solo gruppo**: basta passare il file su chiavetta.

> **PACCHETTI "DI CLASSE"**
> Ogni facilitatore può creare un pacchetto con le domande del proprio gruppo (le ricette del centro anziani, gli orari della biblioteca, i contatti dello sportello del Comune) senza toccare il programma. Il pacchetto si importa sul PC dell'aula e non viene inviato a nessuno.

CAPITOLO

5 Modificare il programma

Dove si trova cosa

File o cartella	Contenuto
`VERSION`, `rilascio.conf`	Numero di versione; indirizzi di pubblicazione
`src/web/js/data-lessons.js`	Le 13 lezioni di base (testo, schemi, quiz)
`src/web/js/data-kb.js`	Le 43 risposte di base della guida interna
`src/contenuti/*.json`	Pacchetti di contenuti inclusi nel programma
`src/web/js/app.js`	Interfaccia: lezioni, domande, voce, impostazioni, ricerca
`src/web/js/board.js`, `practice.js`, `packs.js`	Lavagna; Palestra; pacchetti di contenuti
`src/web/css/app.css`	Aspetto grafico, colori, temi
`src/server/pd_server.py`	Server locale: IA, voce, dati, aggiornamenti
`src/server/pd_packs.py`	Regole dei pacchetti di contenuti
`packaging/`	Avviatore, procedure di configurazione, voci del menu, changelog
`tools/`	Rilascio, pubblicazione, repository APT, versione portatile
`tests/`	Test automatici
`docs/`	Vademecum e guide in PDF (il vademecum viene incluso nel pacchetto)

Provare le modifiche senza installare

```
python3 src/server/pd_server.py --port 8800      # poi aprite http://127.0.0.1:8800 nel browser
PD_DATA_DIR=/tmp/prova python3 src/server/pd_server.py --port 8800   # con dati di prova separati
```

Dopo ogni modifica ai file dell'interfaccia basta ricaricare la pagina (`F5`). Per la parte in Python fermate il server con `Ctrl`+`C` e rilanciatelo.

I test automatici

```
tests/esegui_test.sh
```

In pochi minuti controlla oltre 60 punti: il formato dei pacchetti (anche con esempi sbagliati), tutte le sezioni dell'app, tutti gli schemi, la ricerca nella guida interna su domande tipiche, l'IA simulata, la voce, il microfono, l'importazione dei contenuti, il controllo degli aggiornamenti con un sito simulato, la versione portatile e la resa su telefono. `tools/rilascio.sh` li esegue da solo e **blocca il rilascio** se qualcosa non va, riportando tutto com'era.

> **AGGIUNGETE UNA PROVA PER OGNI ERRORE CORRETTO**
>
> Quando correggete un difetto, aggiungete in `tests/test_ui.py` una riga che lo verifichi: così non tornerà più. Per le nuove domande importanti, aggiungete la frase tipica e la risposta attesa all'elenco `probe`.

Salvare il lavoro con git

```
git status                          # cosa è cambiato
git diff                            # le differenze, riga per riga
git add -A && git commit -m "Nuove domande sulla PEC"
git push                            # copia su GitHub
```

CAPITOLO

6 Pubblicare una nuova versione

Un solo comando prepara tutto. Le note che scrivete tra virgolette compariranno agli utenti nella finestra degli aggiornamenti: scrivetele pensando a loro.

```
tools/rilascio.sh minore \
  "Nuova lezione: la posta certificata (PEC)." \
  "La lavagna salva i disegni anche dopo la chiusura." \
  --apt
```

Cosa fa, nell'ordine

1. Calcola la nuova versione (`correzione`, `minore`, `maggiore` o un numero esplicito) e controlla che sia maggiore dell'attuale.
2. Aggiorna `VERSION` e aggiunge le note in cima a `packaging/changelog`, con data e firma.
3. Esegue **tutti i test**. Se uno fallisce si ferma e ripristina versione e changelog.
4. Compila il pacchetto `.deb`, la versione portatile, l'archivio dei sorgenti; controlla il pacchetto con **lintian**.
5. Prepara `dist/pubblica/` con `aggiornamenti.json` (versione, data, note, impronte SHA-256), le note in Markdown e, con `--apt`, il repository APT firmato.
6. Registra la versione in git (commit e tag `vX.Y.Z`).

La cartella da pubblicare

File	A cosa serve
`aggiornamenti.json`	Letto dal pulsante «Controlla aggiornamenti»
`professore-digitale_X.Y.Z_all.deb`	Il nuovo pacchetto
`professore-digitale-portatile.html`	La versione portatile aggiornata
`contenuti/*.json`	I pacchetti di contenuti scaricabili con un clic
`apt/`	Il repository APT firmato e lo script `installa-repository.sh`
`version.txt`	Compatibilità con il controllo della versione 2.0.0
`NOTE-X.Y.Z.md`, `SHA256SUMS`, vademecum	Note di rilascio, impronte, documentazione

Mettere online

```
tools/pubblica-sito.sh              # carica dist/pubblica/ sul ramo gh-pages di GitHub
git push origin main --tags         # sorgenti e tag, incluso version.txt per chi ha la
2.0.0
```

Dopo 1–2 minuti il sito è aggiornato. Verificate aprendo nel browser `https://NOMEUTENTE.github.io/professore-digitale/aggiornamenti.json`. Facoltativo ma utile: su GitHub, alla voce **Releases**, create una release per il tag con le note e il `.deb` allegato, così chi visita la pagina del progetto trova subito il file.

Pubblicare solo contenuti

1. Modificate il pacchetto in `src/contenuti/` e **aumentatene la versione** (es. 1.0.0 → 1.1.0).
2. `tools/pubblica-contenuti.sh` — valida, copia e aggiorna l'elenco dei contenuti in `aggiornamenti.json`, lasciando invariata la versione del programma.
3. `tools/pubblica-sito.sh` — e un `git commit` per ricordare la modifica.

Gli utenti vedranno **«🔄 Aggiorna (1.0.0 → 1.1.0)»** accanto al pacchetto. Al rilascio successivo del programma il pacchetto aggiornato verrà incluso anche nel `.deb`.

CAPITOLO

7 Come gli utenti ricevono gli aggiornamenti

Tre strade, per tre situazioni diverse. Tutte conservano impostazioni e progressi degli allievi.

A · Dall'app, con un clic (consigliata per i facilitatori)

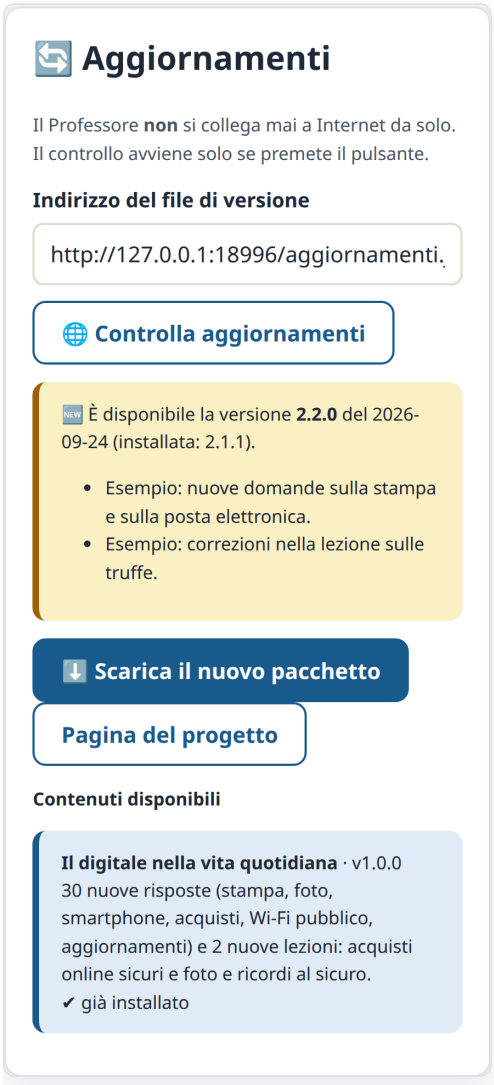


Figura 2. Una nuova versione disponibile, con le note, il pulsante di download e i contenuti.

1. **Impostazioni → Aggiornamenti → Controlla aggiornamenti.** Solo in quel momento il Professore si collega al vostro sito.
2. Se c'è una versione nuova compaiono le **note** che avete scritto.
3. **Scarica il nuovo pacchetto**: il file viene salvato in *Scaricati* solo se la sua impronta SHA-256 corrisponde a quella pubblicata; altrimenti viene scartato.
4. L'app mostra il comando esatto da dare: `sudo apt install ~/Scaricati/professore-digitale_X.Y.Z_all.deb`.
5. I **contenuti** nuovi o aggiornati si installano subito con **Aggiungi** o **Aggiorna**, senza password e senza riavviare.

Il programma non installa da solo il pacchetto di sistema: servirebbero i permessi di amministratore, e la scelta resta alla persona.

B · Dal Gestore aggiornamenti del sistema (consigliata per molti PC)

Con il repository APT firmato, il Professore si aggiorna insieme al resto del sistema: Linux Mint, Ubuntu e Debian lo propongono nel loro **Gestore aggiornamenti**. Su ogni PC basta un comando, una volta sola:

```
curl -fsSL https://NOMEUTENTE.github.io/professore-digitale/apt/installa-repository.sh | sh
```

Lo script aggiunge la chiave pubblica e l'indirizzo del repository, poi installa il Professore. Da quel momento: `sudo apt update && sudo apt upgrade`, oppure il Gestore aggiornamenti grafico. È la strada ideale per i computer rigenerati e donati: ricevono correzioni e novità senza che nessuno debba ricordarsene.

> **PROVATO DAVVERO**
>
> Durante la preparazione di questa guida un computer con la 2.0.0 installata e dei progressi salvati è stato aggiornato alla 2.1.0 tramite il repository APT firmato e poi alla 2.1.1 con il pulsante dell'app. La firma è stata verificata da apt e i progressi sono rimasti intatti.

C · Con la chiavetta (aule senza Internet)

1. Copiate su una chiavetta il nuovo `.deb` e i pacchetti `contenuti/*.json`.
2. Sul PC dell'aula: `sudo apt install ./professore-digitale_X.Y.Z_all.deb`.
3. Per i soli contenuti: Impostazioni → **Importa pacchetto**, oppure `sudo professore-digitale-contenuti installa FILE.json --sistema` per tutti gli utenti del PC.

E chi ha ancora la 2.0.0?

La 2.0.0 sa soltanto leggere un numero di versione da `version.txt` e dire che ce n'è una più recente. `tools/rilascio.sh` genera quel file sia nel sito sia nella radice del progetto su GitHub, dove la 2.0.0 lo cerca. Quegli utenti installano la 2.1 a mano (strada A o C) e da lì in poi hanno tutti gli strumenti nuovi.

CAPITOLO

8 Compatibilità e dati degli allievi

Un aggiornamento riuscito è quello che l'allievo non nota, se non per le novità. I suoi progressi, le sue impostazioni e le sue conversazioni devono sopravvivere a ogni versione.

Dove stanno i dati e chi li tocca

I dati di ogni utente stanno in `~/.local/share/professore-digitale/` (`settings.json`, `progress.json`, `history.json`, `contenuti/`). Il pacchetto `.deb` non li tocca mai, nemmeno con `apt purge`: si cancellano solo con `professore-digitale-pulizia`.

Le regole per chi sviluppa

- **Aggiungere è sempre sicuro**: nuove impostazioni ricevono un valore predefinito (l'elenco `DEFAULTS` in `app.js`).
- **Non rinominare e non cancellare** campi esistenti in una versione minore o di correzione.
- **Non cambiare l'id** di una lezione esistente: i progressi sono legati all'id.
- Se una struttura **deve** cambiare (di solito in una versione maggiore), aumentate `DATA_SCHEMA` in `app.js` e aggiungete un passo nella funzione `migrate()`, che converte i dati vecchi al primo avvio:

```
var DATA_SCHEMA = 2;
function migrate() {
  var p = PD.progress;
  if (!p.schema) p.schema = 1;
  if (p.schema < 2) {            // esempio: dalla 3.0 i progressi sono divisi per allievo
    p.allievi = { "principale": { lessons: p.lessons } };
    p.schema = 2;
  }
}
```

- **Il formato dei pacchetti di contenuti** ha il suo numero (`formato_versione`). Se un giorno servisse cambiarlo, la nuova versione del programma dovrà continuare a leggere anche il formato 1.
- **Provate sempre l'aggiornamento**, non solo l'installazione pulita: installate la versione precedente, create dei progressi, aggiornate e controllate che ci siano ancora.

Tornare indietro

Se una versione pubblicata ha un problema serio, non ritiratela: pubblicate subito una **correzione**. Nel frattempo un utente può reinstallare la versione precedente dalla cartella `archivio/` o dal repository APT: `sudo apt install professore-digitale=2.1.0 --allow-downgrades`. I dati restano al loro posto.

CAPITOLO

9 Migliorie consigliate fino alla 3.0

Una tabella di marcia ragionata, pensata per i contesti reali del progetto: centri anziani, aule senza rete, computer ricondizionati, facilitatori volontari. Le priorità tengono conto dell'utilità per gli allievi e dell'impegno richiesto.

Versione 2.2 · Il facilitatore al centro

Miglioria	Perché	Priorità
Editor grafico dei contenuti	Un modulo nell'app per scrivere domande e lezioni con anteprima dello schema, senza toccare il JSON. Apre la creazione dei contenuti a tutti i facilitatori.	Alta
Dispense stampabili	Ogni lezione in una pagina A4 con testo grande, schema e spazio per gli appunti: molti allievi imparano meglio con la carta.	Alta
Attestato di partecipazione	Un PDF con nome e lezioni completate: motivazione e riconoscimento per chi finisce il percorso.	Media
Esportazione di una conversazione	Salvare le risposte utili in PDF da portare a casa.	Media

Versione 2.3 · Più allievi, più voce

Profili allievo	Più persone sullo stesso PC, ognuna con i propri progressi, scegliendo il proprio nome all'avvio. (Richiede la migrazione dei dati: vedi capitolo 8.)	Alta
Voce naturale di serie	Proporre Piper già durante la prima configurazione: la voce robotica è il limite più sentito dagli anziani.	Media
Ripasso intelligente	Riproporre dopo qualche giorno le domande di quiz sbagliate.	Media
Statistiche per il facilitatore	Quali lezioni sono più difficili, quali domande vengono fatte più spesso (tutto locale, anonimo).	Bassa

Versione 3.0 · La Cattedra in rete

Modalità aula	Il PC del facilitatore guida i PC degli allievi sulla rete locale: stessa lezione su tutti gli schermi, quiz collettivi, "alzata di mano" digitale. Sempre senza Internet.	Alta

Documenti locali per l'IA	L'IA risponde anche usando PDF scelti dal facilitatore (regolamenti del Comune, orari, guide dei servizi), con la stessa tecnica già usata per la guida interna.	Media
Più lingue	Interfaccia e contenuti traducibili, per i corsi di italiano per stranieri e le comunità multilingue.	Media
Integrazione con FeniceOS	Professore Digitale preinstallato e configurato nella distribuzione live, con modelli vocali e contenuti già inclusi per l'uso completamente offline.	Alta

Nuovi pacchetti di contenuti consigliati

Sono i temi che emergono più spesso nei corsi di facilitazione digitale. Ognuno può diventare un pacchetto da 10–30 domande e una lezione, senza nuove versioni del programma:

- **Sanità digitale**: fascicolo sanitario, prenotazioni, referti online, ricette elettroniche
- **PEC e firma digitale**
- **Social network con giudizio**: privacy, notizie false, foto dei nipoti
- **Intelligenza artificiale generativa**: usarla, riconoscere immagini e voci false
- **Pagamenti**: bonifici istantanei, pagamenti con il telefono, SPID per la banca
- **Trasporti e viaggi**: biglietti online, mappe, app dei treni
- **Casa digitale**: TV connessa, assistenti vocali, bollette online
- **Lavoro e studio**: curriculum, videocolloqui, documenti condivisi
- **Glossario esteso**: 100 parole del digitale spiegate in una frase
- **Pacchetti locali**: sportelli e servizi del proprio territorio

ATTENZIONE AI CONTENUTI CHE INVECCHIANO

Procedure della Pubblica Amministrazione, costi e nomi delle app cambiano spesso. Mettete questi argomenti in pacchetti separati, con la data nella `descrizione`, e rivedeteli ogni anno: aggiornare un pacchetto è questione di minuti.

Distribuzione e comunità

- **Sito del progetto** su GitHub Pages con pagina di download, vademecum e pacchetti di contenuti in un unico posto.
- **Contributi dai facilitatori**: raccogliete i pacchetti delle associazioni (via email o GitHub), validateli con `professore-digitale-contenuti valida` e pubblicate i migliori.
- **Flatpak** (dalla 3.0): per arrivare anche alle distribuzioni non basate su Debian.

CAPITOLO

10 Lista di controllo e problemi frequenti

Prima di ogni rilascio

- ☐ Ho deciso il tipo di versione (correzione, minore, maggiore) con le regole del capitolo 2.
- ☐ Le note sono scritte per gli utenti, in italiano semplice, una novità per nota.
- ☐ Ho aggiunto un test per ogni errore corretto e per le nuove domande importanti.
- ☐ Ho aumentato la `versione` dei pacchetti di contenuti modificati.
- ☐ `tests/esegui_test.sh` passa senza errori.
- ☐ Ho provato l'**aggiornamento** dalla versione precedente su un PC di prova, con progressi salvati.
- ☐ Se il vademecum è cambiato, il PDF in `docs/` è aggiornato.
- ☐ Dopo `tools/pubblica-sito.sh` ho aperto `aggiornamenti.json` nel browser e ho premuto «Controlla aggiornamenti» da un PC.
- ☐ Ho fatto `git push origin main --tags`.

Problemi frequenti

Messaggio o sintomo	Soluzione
«La nuova versione deve essere maggiore di quella attuale»	Il numero indicato è già stato usato. Usate `correzione` / `minore` / `maggiore` invece del numero esplicito.
«I test non sono passati»	Leggete le righe `FAIL`: dicono cosa non funziona e cosa ci si aspettava. Correggete e rilanciate lo stesso comando: versione e changelog sono già stati riportati indietro.
Un pacchetto viene «ignorato perché non valido»	`professore-digitale-contenuti valida FILE.json` indica campo e posizione. Errori tipici: virgola finale prima di `]` o `}`, virgolette non chiuse, `corretta` contata da 1 invece che da 0.
«serve il Professore Digitale X o successivo»	Il pacchetto ha `richiede_app` più alto della versione installata: aggiornate il programma o abbassate il valore se il pacchetto non usa funzioni nuove.
Una nuova domanda non viene trovata	Aggiungete parole chiave con le parole che usano gli allievi (anche sbagliate o dialettali). Ogni parola chiave pesa più del testo della risposta.
Lo schema compare come testo	Il codice Mermaid ha un errore. Provatelo su `mermaid.live`, usate le virgolette nei riquadri e ricordate `\n` e `\"` nel JSON.
«impronta SHA-256 non corrispondente»	Il file sul sito non è quello descritto in `aggiornamenti.json` (caricamento incompleto o misto). Ricaricate tutta la cartella `dist/pubblica/`.
apt: «NO_PUBKEY» o «not signed»	Manca la chiave pubblica sul PC: rilanciate `installa-repository.sh`. Se avete cambiato chiave di firma, tutti i PC devono reinstallarla.

Messaggio o sintomo	Soluzione
Il sito non si aggiorna	GitHub Pages impiega 1–2 minuti; ricaricate con `Ctrl` + `F5`. Controllate in Settings → Pages che la sorgente sia il ramo `gh-pages`.
`pubblica-sito.sh` chiede la password	GitHub non accetta più la password dell'account: create un *Personal access token* (Settings → Developer settings) o configurate una chiave SSH.

I comandi in breve

`tests/esegui_test.sh`	Tutti i test
`./build.sh`	Compila la versione scritta in `VERSION` (senza rilascio)
`tools/rilascio.sh TIPO "note" … [--apt]`	Nuova versione completa
`tools/pubblica-contenuti.sh`	Aggiorna solo i contenuti pubblicati
`tools/pubblica-sito.sh`	Carica `dist/pubblica/` su GitHub Pages
`tools/repo-apt.sh --crea-chiave`	Crea la chiave di firma (una volta)
`professore-digitale-contenuti valida|installa|elenco|rimuovi|modello`	Gestione dei pacchetti di contenuti



Un progetto che cresce con chi lo usa.

Ogni domanda aggiunta da un facilitatore, ogni correzione, ogni computer rigenerato rende la Cattedra Digitale più utile per la persona successiva.

Aggiornare con cura, verificare sempre, rispettare le scelte di chi impara: è la tecnologia gentile applicata anche alla manutenzione del software.

Danilo D'Auria
Tecnico IT & Divulgatore Digitale per il Sociale
danilodauria.site@gmail.com

Professore Digitale 2.1.1
Licenza MIT · Settembre 2026